

Planning Tool for an Autonomous Drone Delivery Network in Nova Silda

Documentation

Subject: Algorithms and Data Structures - Winter Semester 2025/2026
Group members: Radek Bäuml - s241229, Yavuz Ceyhun Bayram - s240625

Table of Contents

Table of Contents	1
Project Overview	2
System Architecture	2
Core Components	2
Node Types	3
Edge Properties	3
Implemented Functions	3
B1: Import Drone Network	3
B2: Define No-Fly Zones	4
B3: Extend/Modify Network	4
F1: Check Reachability	5
F2: Efficient Flight Routes	6
F3: Delivery Capacity	6
F4: Network Resilience	7
F6: Communication Infrastructure	8
Complexity Analysis	9
Summary Table	9
Notation	9
Usage Examples	10
Command-Line Interface	10
Available Commands	10
Conclusion	11

Project Overview

The city of Nova Schilda plans to introduce an automated delivery system using autonomous drones. The goal of this project is to design and implement a planning tool that centrally manages, analyzes, and optimizes the drone delivery network. The network consists of distribution hubs, delivery points, charging stations, and flight corridors with various constraints.

This project was implemented in **Python** and is based on classical graph data structures and algorithms, as taught in the course.

System Architecture

Core Components

graph.py - Graph data structure implementation

- `Edge` class: Represents flight corridors with properties (energy cost, capacity, distance, restriction status)
- `Graph` class: Directed graph using adjacency list representation

network_io.py - Network import/export functionality

- `load_graph_from_json()`: Loads network from JSON files

modify_network.py - Network modification operations

- `set_no_fly_zone()`: Marks corridors as restricted
- `clear_no_fly_zone()`: Removes restrictions

reachability.py - Connectivity analysis

- `reachable_from_hub()`: Verifies all delivery points are accessible

routing.py - Path finding algorithms

- `dijkstra()`: Computes shortest paths based on energy cost

capacity.py - Flow analysis

- `edmonds_karp()`: Calculates maximum flow/delivery capacity

resilience.py - Network reliability analysis

- `find_min_cut_edges()`: Finds minimum cut set
- `find_bottleneck_edges()`: Identifies fully utilized edges

communication.py - Infrastructure optimization

- `compute_communication_network()` : Builds minimum spanning tree

main.py - Command-line interface

Node Types

- **hub**: Distribution centers
 - **delivery**: End delivery points
 - **charging**: Charging stations
 - **relay**: Relay/intermediate stations
-

Edge Properties

- **energy_cost**: Energy consumption for traversal
 - **capacity**: Maximum simultaneous drones (per hour)
 - **distance**: Physical distance
 - **bidirectional**: Whether edge works both directions
 - **restricted**: No-fly zone status
-

Implemented Functions

B1: Import Drone Network

- **Function:** `load_graph_from_json()`

Purpose: Reads and constructs the drone network from a JSON file

Problem Type: Graph construction from external data

Algorithm Choice: Direct graph construction

- Read nodes first to ensure all vertices exist
- Add edges with all properties
- Handle bidirectional edges by creating two directed edges

Data Structures:

- **Dictionary (node_types):** O(1) lookup for node type verification
- **Dictionary of lists (adjacency_list):** O(1) access to outgoing edges

Complexity:

- **Time:** $O(N + E)$ where N = number of nodes, E = number of edges
 - **Space:** $O(N + E)$ for storing the graph
-

B2: Define No-Fly Zones**Functions:**

- `set_no_fly_zone()`
- `clear_no_fly_zone()`

Purpose: Mark or unmark flight corridors as restricted due to weather, regulations, or disruptions

Problem Type: Graph edge property modification

Modeling Decisions:

- No-fly zones are represented as a boolean flag on edges rather than removing edges entirely

Algorithm Choice: Direct edge property update

- Iterate through adjacency list of source node
- Find matching destination and toggle restriction flag

Data Structures:

- Adjacency list traversal

Complexity:

- **Time:** $O(k)$ where k is the number of edges leaving the node
 - **Space:** $O(1)$ - in-place modification
-

B3: Extend/Modify Network**Functions:**

- `Graph.add_node()`
- `Graph.add_edge()`

Purpose: Dynamically extend the network with new nodes or corridors

Problem Type: Dynamic graph modification

Modeling Decisions:

- New nodes can be added without affecting existing structure
- New edges automatically handle bidirectionality

Algorithm Choice: Direct insertion

- For nodes: Add to `node_types` dictionary and create empty adjacency list
- For edges: Append Edge object to adjacency list

Data Structures:

- Dictionary operations for $O(1)$ insertion

Complexity:

- **Time:** $O(1)$ for both `add_node` and `add_edge`
 - **Space:** $O(1)$ per new element
-

F1: Check Reachability

Function: `reachable_from_hub()`

Purpose: Verify that all delivery points can be reached from the distribution hub, respecting no-fly zones

Problem Type: Graph connectivity/reachability problem

Modeling Decisions:

- Only delivery nodes need to be reachable (not all nodes)
- Restricted edges (no-fly zones) are not traversable
- Uses unweighted traversal since we only care about connectivity, not path cost

Algorithm Choice: Breadth-First Search (BFS)

- BFS guarantees finding all reachable nodes

Alternative Approaches Considered:

- DFS would also work but both DFS and BFS have same complexity

Data Structures:

- **Queue (deque):** $O(1)$ append and popleft operations for BFS
- **Set (visited_nodes):** $O(1)$ membership testing

Complexity:

- **Time:** $O(V + E)$ where V = vertices, E = edges
 - **Space:** $O(V)$ for visited set and queue
-

F2: Efficient Flight Routes

Function: `dijkstra()`

Purpose: Find the most energy-efficient route from a hub to a delivery location, avoiding no-fly zones

Problem Type: Single-source shortest path problem with non-negative weights

Modeling Decisions:

- Edge weight = energy_cost (optimizing for battery consumption)
- Restricted edges are skipped entirely
- Returns both path and total cost

Algorithm Choice: Dijkstra's Algorithm

- Optimal for non-negative weights (energy costs are always ≥ 0)
- Guarantees shortest path

Data Structures:

- **Min-heap (priority queue):** $O(\log V)$ insertions and deletions
- **Dictionary (dist):** $O(1)$ distance lookups
- **Dictionary (previous):** $O(1)$ path reconstruction

Complexity:

- **Time:** $O((V + E) \log V)$ with binary heap
 - **Space:** $O(V)$ for distance and predecessor maps
-

F3: Delivery Capacity

Function: `edmonds_karp()`

Purpose: Calculate maximum number of drones that can simultaneously fly from a hub to a delivery area without exceeding corridor capacity limits

Problem Type: Maximum flow problem

Modeling Decisions:

- Edge capacity represents max simultaneous drones per hour

- Flow represents actual drone traffic
- Restricted edges have zero capacity (excluded from flow)
- Returns both max flow value and residual capacity map

Algorithm Choice: Edmonds-Karp

- Guaranteed $O(VE^2)$ termination
- BFS ensures shortest augmenting paths

Data Structures:

- **Nested defaultdict (capacity_map):** Residual graph with $O(1)$ access
- **Queue (deque):** BFS traversal
- **Dictionary (parent):** Path reconstruction

Algorithm Details:

1. Build residual capacity graph
2. Repeatedly find augmenting paths using BFS
3. Compute bottleneck capacity along path
4. Update residual capacities (subtract forward, add backward)
5. Accumulate flow

Complexity:

- **Time:** $O(V \cdot E^2)$ - BFS is $O(V+E)$, repeated at most $O(VE)$ times
- **Space:** $O(V + E)$ for residual graph

F4: Network Resilience

Functions:

- `find_bottleneck_edges()`
- `find_min_cut_edges()`

Purpose: Identify critical network components whose failure would severely impact delivery capacity.

Problem Type: Network reliability analysis / Min-cut problem

Modeling Decisions:

- **Bottlenecks:** Edges operating at full capacity in max-flow
- **Min-cut:** Minimum set of edges whose removal disconnects source from sink
- Uses residual graph from max-flow computation

Algorithm Choice:

- Build on Edmonds-Karp results
- Min-cut via max-flow/min-cut theorem

Min-Cut Algorithm:

1. Run BFS on residual graph from source
2. Find all nodes reachable via positive residual capacity
3. Min-cut edges go from reachable to unreachable nodes

Bottleneck Algorithm:

1. Iterate through all edges
2. Check if residual capacity is zero (fully saturated)
3. Original capacity > 0 ensures it's a real edge

Data Structures:

- **Set (reachable):** $O(1)$ membership testing
- **Queue (deque):** BFS traversal

Complexity:

- **Min-cut:** $O(V + E)$ - single BFS
- **Bottlenecks:** $O(E)$ - edge iteration
- **Total:** $O(V + E)$ after max-flow is computed

F6: Communication Infrastructure

Function: `compute_communication_network()`

Purpose: Establish a minimum-cost radio communication network connecting all drone stations.

Problem Type: Minimum Spanning Tree (MST)

Modeling Decisions:

- Weight = edge capacity
- Must connect all stations with minimum total cost
- Creates undirected connectivity structure
- Restricted edges are excluded

Algorithm Choice: Prim's Algorithm

- Starts from arbitrary node and grows MST greedily
- Efficient with priority queue: $O((V+E) \log V)$
- Good cache locality for dense graphs

Data Structures:

- **Min-heap (priority_queue):** Stores edges by weight
- **Set (visited_node_ids):** $O(1)$ membership check
- **List (selected_edges):** Stores MST edges

Algorithm Details:

1. Start from arbitrary node
2. Add all outgoing unrestricted edges to priority queue
3. Repeatedly select minimum-weight edge to unvisited node
4. Add node to visited set and add its edges to queue
5. Continue until all nodes visited

Complexity:

- **Time:** $O((V + E) \log V)$ with binary heap
 - **Space:** $O(V + E)$ for heap and visited set
-

Complexity Analysis

Summary Table

Function	Time Complexity	Space Complexity	Algorithm
Load Graph	$O(N + E)$	$O(N + E)$	Direct construction
No-Fly Zone	$O(k)$	$O(1)$	Edge update
Reachability	$O(V + E)$	$O(V)$	BFS
Routing	$O((V+E) \log V)$	$O(V)$	Dijkstra
Capacity	$O(V \cdot E^2)$	$O(V + E)$	Edmonds-Karp
Bottlenecks	$O(E)$	$O(1)$	Edge iteration
Min-Cut	$O(V + E)$	$O(V)$	BFS on residual
Communication	$O((V+E) \log V)$	$O(V + E)$	Prim's MST

Notation

- **V:** Number of vertices/nodes
- **E:** Number of edges
- **k:** the number of edges leaving the node

Usage Examples

Command-Line Interface

The system includes a test console in **main.py**:

Available Commands

reach <hub>

Example: reach HUB

Output: All nodes reachable: True

route <start> <target>

Example: route HUB D2

Output:

Path: HUB -> CH1 -> D2

Cost: 11.0

flow <source> <sink>

Example: flow HUB D3

Output: Maximum flow: 3

resilience <source> <sink>

Example: resilience HUB D3

Output:

Max flow: 3

Bottlenecks: [('D1', 'D2'), ('D2', 'D3'), ('CH1', 'R1'), ('R1', 'D3')]

Min-cut edges: [('R1', 'D3'), ('D2', 'D3')]

mst

Example: mst

Output:

Total cost: 9

HUB -- CH2 (2)

CH2 -- R1 (1)

HUB -- D1 (2)

D1 -- D2 (1)

D2 -- D3 (1)

R1 -- CH1 (2)

addnode <id> <type>

Example: addnode D5 delivery

Output: Node 'D5' added as type 'delivery'

addedge <from> <to> <energy_cost> <capacity> [distance] [bidirectional] [restricted]

Example: addedge D5 D1 6 3 5 false true

Output: Edge added: D5 -> D1 (energy: 6, capacity: 3, bidirectional: False, distance: 5.0, restricted: True)

nofly <u> <v>

Example: nofly HUB D1

Output: No-fly zone added: True

clear <u> <v> [true]

Example: clear HUB D1

Output: No-fly zone removed: True

Conclusion

This planning tool successfully implements core graph algorithms for drone network management. The system efficiently handles routing optimization, capacity analysis, and infrastructure planning through chosen algorithms and data structures. The modular design allows for easy extension and maintenance.